Certificate Supplement to Improved Upper Bounds on the Size of Condorcet Domains

Pavlos Moros Søren Riis

Local review draft, 29 October 2026

S1 Scope and notation

This supplement records the finite lemma, its certificate and exact reproduction instructions. The general proof remains in the main paper. Certificate files are unchanged from the frozen 29 October source; no finite generation or audit was rerun for this editorial preparation.

Let $\mathcal{F}_k$ consist of the nonempty families of orders on k alternatives defined by one extreme never-condition on each triple and arbitrary fixed pair comparisons. A condition $x\mathrm{N}p$ means that x never has position p on its triple; extreme means $p = 1$ or $p = 3$. Write $A(H)$ for the number of enabled adjacent swaps, counted over orders and positions, so each undirected edge contributes twice. The definitions below reproduce the exact finite argument used in the manuscript, with additional enumeration and selector details.

S2 A finite window inequality

For $H \in \mathcal{F}_k$, its *bottom slices* group orders by their last alternative and delete it; top slices group by the first alternative. Every nonempty slice belongs to $\mathcal{F}_{k-1}$. Write $a_j(H)$ for the number of orders whose swap at position j is enabled. On seven labels put

$$C(H) = \frac{a_1 + 2a_2 + 2a_3 + 2a_4 + 2a_5 + a_6}{10}. \tag{1}$$

The weights sum to one. They are chosen so that each interior swap is counted with total weight one when a seven-position window slides through an order.

Lemma S2.1 (Exact finite estimate)**.** *There is a rational function q on the isomorphism types of $\mathcal{F}_6$ such that, for every $J \in \mathcal{F}_7$,*

$$C(J) \le \frac{5}{8}|J| + \sum_{H \text{ bottom slice}} q(H) - \sum_{H \text{ top slice}} q(H). \tag{2}$$

The rational table and exact verifiers are in `ancillary/finite_certificate/`*. The finite statement includes all fixed-pair constraints, with no common-axis assumption.*

Proof. Here is the construction and the coverage argument for the finite check. The certificate assigns per-order weights ϕ to five-label types. For a six-domain H let $b(H)$ and $t(H)$ be the sums of $|L|\phi(L)$ over its bottom and top five-slices. Put

$$s(H) = \frac{12}{19}|H| + b(H) - t(H) - \frac{1}{5}A(H), \qquad \eta = \frac{1}{152}.$$

There are specified exceptional sets $\mathcal{T}, \mathcal{S}$ of five and nine six-label types. Put $p(H) = 2\eta(r_t(H) - r_b(H))$ outside $\mathcal{S}$, and $p(H) = \psi(H)$ on $\mathcal{S}$, with the rational values ψ supplied by the certificate. Set $\theta_b = s/2 + p$ and $\theta_t = s/2 - p$. The finite facts are as follows. On $\mathcal{T}$ both split values are zero. Outside $\mathcal{T} \cup \mathcal{S}$,

$$\theta_b(H) \geq \frac{\eta}{2}|H| + 4\eta r_t(H), \qquad \theta_t(H) \geq \frac{\eta}{2}|H| + 4\eta r_b(H). \tag{3}$$

Here r_b, r_t count specified five-slices described below. A further finite check proves

$$\Lambda(J) := \sum_{\text{bottom } H} \theta_b(H) + \sum_{\text{top } H} \theta_t(H) \geq \eta|J| \tag{4}$$

whenever J has a slice in $(\mathcal{T} \setminus \{R_8\}) \cup \mathcal{S}$. All types, weights and slice selectors are explicitly defined in the ancillary data; they are not fitted during verification.

For completeness, the one remaining exceptional type is

$$R_8 = \{524130, 524310, 534120, 534210, 542130, 542310, 543120, 543210\}.$$

It has a common first and last alternative. In the regenerated five-label class map the two selector types below have indices 866 and 41, respectively. The quantity $r_b(H)$ counts bottom five-slices of the type R_8 with its first alternative deleted; $r_t(H)$ counts top five-slices of the type with its last deleted. Both counts vanish on R_8. If a bottom R_8 slice occurs, its eight orders share their first alternative, so they enter one top receiver slice and contribute one to its r_b. That receiver is not R_8. Distinct bottom slices give distinct receiver/label pairs. Each contributes 4η, exactly paying the missing baseline $8\eta/2$. Top R_8 slices are symmetric. In the remaining cases all non-R_8 slices lie outside $\mathcal{T} \cup \mathcal{S}$, so (3) applies to every receiver. It proves (4) in every case not handled by the exceptional finite check.

Define

$$q(H) = \frac{b(H) + t(H)}{2} + p(H).$$

In a seven-domain, a top slice of a bottom slice is the same conditional five-domain as the corresponding bottom slice of a top slice. These middle terms cancel. Also the sum of $A(H)$ over all six-slices is $10C(J)$. Consequently

$$\Lambda(J) = \frac{12}{19}|J| - C(J) + \sum_{\text{bottom } H} q(H) - \sum_{\text{top } H} q(H).$$

Now (4) and $12/19 - 1/152 = 5/8$ give (2).

We finally explain why the finite checks are exhaustive. All 128 400 labelled five-domains form 1 160 relabelling types. Extend their defining laws over the ten new triples and five new pairs. This gives 14 105 084 nonempty six-extensions of five-type representatives and 94 830 feature rows. The extension count is not a count of distinct labelled six-domains. Every six-domain occurs up to relabelling: restrict its *defining conditions* to five labels, rather than taking its projection. Keeping every domain on the thirteen exceptional feature rows gives precisely the fourteen required types. Two of the five types in $\mathcal{T}$ share one feature row. Their representatives and reversals, omitting R_8, give eighteen seven-label seed families. Requiring a prescribed endpoint six-slice and intersecting all compatible law and pair options produces 14 248 seed–domain cases (before deduplication across seeds) and 18 316 feature rows, covering both orientations. Every required inequality is verified by clearing the rational denominators and checking a nonnegative integer residual. A separate tuple-set checker recomputes every saved row from its witness orders. The written coverage and receiver arguments are given above. The separate Lean project in the repository also formalises these steps and the passage to arbitrary n; its verification report records the checked scope. □

S3 Reproducing the finite checks

The repository directory `ancillary/finite_certificate/` contains standard Python 3 scripts, two C++17 generator sources, rational certificate data, saved feature rows and tuple witnesses. No optimizer, network access or nonstandard Python package is needed. Run commands from that directory.

Saved-data replay.

```
sh run_joint_checks.sh
```

This checks the rational residuals on saved rows, recomputes features from all 94 830+18 316 = 113 146 tuple witnesses, checks the compressed potential and scalar transfers, and runs supplementary small charging-map cases. Successful completion has exit status zero. It checks the supplied data, rather than reconstructing the family coverage.

Complete regeneration. With a C++17 compiler named `c++` available, use a new output path:

```
python3 run_fresh_regeneration.py /path/to/new/output
```

The driver refuses to overwrite an existing output directory. It starts from generator sources, coefficient data and exceptional keys, not saved feature rows or witnesses. It regenerates the five-label families, six-extensions and required seven-label families, checks completion flags, runs the exact and tuple verifiers, and compares regenerated feature tables with the supplied tables. Its success receipt is `FULL_REGENERATION_RECEIPT.json`, with result `COMPLETE FRESH REGENERATION PASS`. Each command and return code is recorded there. Witness order can depend on hash iteration; feature rows are compared exactly, and every regenerated witness is checked.

Expected counts. The units are different and should not be interchanged:

- 128 400 labelled five-families and 1 160 relabelling types;
- 14 105 084 six-extensions and 94 830 six-feature rows;
- 397 selected sets, giving fourteen exceptional types on thirteen feature rows;
- eighteen seven-label seeds, 14 248 seed–domain cases before cross-seed deduplication, and 18 316 seven-feature rows.

Both orientations are covered. The generic and R_8 receiver cases are the hand arguments in Section S2; they are not replaced by a success flag. The separate Lean project formalises the coverage proof and the passage from seven positions to arbitrary n; it is documented in the repository.

S4 Verification

The repository records the prior complete finite regeneration and a fresh saved-data replay. Lean sources, documentation and verification scope are provided separately through the GitHub repository and archived release referenced in the main paper.